ENGINEERING REFERENCE

Technical Documentation

Architecture, APIs, database schema, integrations, and deployment guide for the LogisX dispatch & fleet management platform.

LOGISX INC. · **DESHORN KING, CEO**

MAY 12, 2026

Contents

Overview

A high-level introduction to the LogisX platform — what it does, who uses it, and how the pieces fit together.

What is LogisX

LogisX is a dispatch and fleet management platform for small trucking companies. It coordinates the movement of freight from broker to delivery: dispatchers assign loads to drivers, drivers report their position and status from the road, investors who own the trucks see real-time earnings on the fleet they fund, and customers can track their load with a public link.

The system is a Node.js / Express backend, a Vue 3 single-page application for the user-facing surfaces, and Google Sheets as the system of record for job tracking — with SQLite holding everything that doesn't fit the Sheets model (users, sessions, geocode cache, expenses, applications, onboarding documents, telemetry).

Who uses it

Four roles, each with a tailored experience.

ROLE	WHAT THEY DO	PRIMARY SURFACE
Super Admin	Operations leadership. Manages users, reviews applications, approves invoices, sees full financials.	Dashboard + every admin route
Dispatcher	Daily operations. Assigns loads, tracks trucks live, handles driver communication.	Dashboard, tracking map, messages
Driver	The person in the truck. Accepts loads, reports GPS, updates status, uploads POD, logs expenses.	Mobile-optimized `/driver` view
Investor	The owner-operator who funded a truck. Sees earnings, fleet performance, and signed documents.	`/investor` dashboard

In addition, three public flows require no login: driver applications at `/apply`, investor applications at `/invest`, and the customer load tracker at `/track/:loadId`.

Stack at a glance

- **Backend** — Node.js, Express, Socket.IO, better-sqlite3, googleapis (Sheets/Drive), Puppeteer (HTML→PDF), pdfkit, pdf-lib, nodemailer.
- **Frontend** — Vue 3, Vite, Vue Router, Pinia, Tailwind CSS v4, shadcn-vue, Vant (mobile), Leaflet + Google Maps.
- **Storage** — Google Sheets ("Job Tracking" tab is the source of truth for loads), SQLite (`app.db`, WAL mode, ~35 tables for everything else).
- **Real-time** — Socket.IO rooms per role (dispatch, investor, driver:<name>) for instant load assignments, status updates, and dashboard refreshes.
- **Integrations** — Google Sheets, Google Drive (POD uploads), Google Maps (geocoding, routing, weather), Gemini 2.5 Flash (receipt OCR), n8n (broker email ingestion), Routemate (ELD telematics, off by default), Tesseract (legacy POD OCR).

How loads move through the system

```
Broker email arrives
    └─► n8n parses the load details and posts to /api/n8n/job
            └─► row appended to "Job Tracking" sheet (status: Unassigned)
                    └─► Dispatcher assigns driver via /dashboard
                            └─► Socket.IO emits "load-assigned" to the driver
                                    └─► Driver accepts → status: Dispatched
                                            └─► GPS reports → geofence auto-advances
status
                                                    └─► Driver uploads POD → status:
Delivered
                                                            └─► Super Admin generates
invoice
                                                                    └─► Invoice
approved → driver paid
```

Every step is logged: socket events fan out to the dispatcher's UI, status changes write to the "Status Logs" sheet, and admin-significant actions land in the `audit_trail` SQLite table.

What this document covers

The chapters that follow are organized by concern: backend, frontend, integrations, document generation, deployment, and operations. The final appendix is a flat API reference of every route in the system. Each chapter is self-contained — read top-to-bottom for a complete tour, or jump to the topic you need.

If you are looking for how *users* perform tasks (sign onboarding documents, accept a load, generate an investor report), see the companion **User Manual** PDF instead.

Backend

The backend lives in a single file, `server.js`, ~12,000 lines and growing. It exposes roughly 150 REST endpoints, hosts a Socket.IO server for real-time fan-out, and brokers between three storage layers: SQLite (`app.db`), Google Sheets, and Google Drive.

Why a single file

When LogisX started, the data model lived entirely in Google Sheets and the server was a thin proxy. As features piled on — SQLite tables, real-time sockets, PDF generation, OCR, telematics — the file kept growing rather than fragmenting. The trade-off is real: one file is easy to grep but hard to refactor. We keep it intentionally for now because:

- The route handlers are mostly independent (no deep shared state beyond a few singletons).
- Logical sections are clearly fenced with comment banners.
- Splitting into modules would require designing the boundaries that don't naturally exist yet.

When a domain finally outgrows the single-file model (the PDF rendering already has, hence `lib/pdf-browser.js`), we lift it into `lib/` as a focused module. Anything in `lib/` is the right place to look first for self-contained logic.

Process model

`server.js` boots in this order:

1. Load `.env` via `dotenv`.
2. Open `app.db` via `better-sqlite3` (synchronous, WAL-mode SQLite). All queries are synchronous — no connection pool, no async lock.
3. Run schema migrations (described below).
4. Construct Express app: body parser (50 MB limit for base64 photos), compression, session middleware (SQLite-backed), CSP-free static serving from `client/dist/` (or `public/` fallback).
5. Mount routes.
6. Wire Socket.IO onto the same HTTP server.

7. Start background intervals (driver-location cleanup, Routemate sync if enabled, demo-viewer write lockdown ticks).
8. Listen on `process.env.PORT` (default 3000).

A single Node process handles everything — there is no worker pool, no queue, no sidecar. The 60-second in-memory Sheets cache (`getJobTrackingCached()`) is the only thing keeping this honest under load.

Storage layout

The system has three storage backends serving different needs.

Google Sheets (system of record for loads)

The "Job Tracking" tab in spreadsheet `1ey1n0AAG0k8k-qwkWh2T_C8VqqY129OQQr7D5wNl7Mo` is the canonical source for every load that ever moved through LogisX. Columns include Load ID, Driver, Status, Pickup / Drop-off Appointment, Payment, Owner ID, and many more — the exact schema is determined by header row 1 and discovered by regex at query time, not by a fixed type definition.

Why Sheets and not a database? History: clients started in Sheets and want to keep the same interface. n8n also reads/writes the sheet directly when ingesting broker emails, so the sheet is the integration point. Sheets imposes a 300 req/min quota, which is the reason for the 60-second in-memory read cache.

A separate archive spreadsheet (`1WCiMmcI7GuS4eFaG9PAop5CFtMKKtfla1sOAKxcEduI`) holds historical data accessed only via `/archive`.

SQLite (`app.db`, WAL mode)

About 35 tables holding everything that doesn't fit the Sheets model. Grouped by domain:

DOMAIN	TABLES
Auth & users	`users`, session store
Messaging	`messages`, `notifications`, `dispatch_notifications`
Dispatch state	`load_responses`, `load_ratings`, `load_coordinates`, `deleted_loads`
Fleet	`trucks`, `truck_assignments`, `trailers`, `drivers_directory`, `carrier_driver_history`
Finance	`expenses`, `maintenance_fund`, `compliance_fees`, `invoices`, `investor_config`
Investors	`investors`, `investor_applications`, `investor_onboarding`, `investor_onboarding_documents`, `investor_payment_info`, `investor_outreach_log`
Driver onboarding	`job_applications`, `driver_onboarding`, `onboarding_documents`, `driver_payment_info`
Documents	`documents`, `legal_documents`
Tracking	`driver_locations`, `geocode_cache`
Telematics	`routemate_vehicles`, `routemate_telemetry`, `routemate_fault_codes`, `routemate_dvir`, `routemate_fuel_daily`, `routemate_hos_daily`
Admin	`audit_trail`

The SQLite file lives at the project root and is the only thing other than `.env` and `service-account-key.json` that needs to be backed up.

Google Drive (POD storage)

Driver-uploaded proof-of-delivery photos are converted to PDF (via `imageToPdf()` in `server.js`) and uploaded to a single shared Drive folder identified by `GOOGLE_DRIVE_FOLDER_ID`. The Drive file ID is stored back in SQLite (`documents.drive_file_id`). If the environment variable is unset, uploads silently skip Drive and persist only locally.

Schema migrations

There is no migration framework. Two patterns coexist:

Additive column adds. On boot, the server tries `ALTER TABLE … ADD COLUMN` for every column that may not yet exist, wrapped in `try/catch`. The catch absorbs the duplicate-column error and moves on. This is the path used for the vast majority of schema changes.

Rename-recreate. When a CHECK constraint changes (most commonly the `users.role` constraint when a new role is added), the migration tries a test insert in `try/catch`; on failure, it renames the old table, creates the new one with the updated constraint, copies data over, drops the renamed table, and clears all active sessions so users are forced to re-login. This is more invasive but reversible at the cost of a forced re-login.

Migrations are idempotent and run on every boot. A fresh `app.db` and a long-lived `app.db` reach the same final schema.

Authentication & authorization

Session-based, no JWTs. Cookies are `httpOnly`, `secure` in production, `sameSite=strict`. The session secret comes from `SESSION_SECRET` in `.env` — in development there is a default fallback, but production must set it.

Four roles enforced by two middlewares:

- `requireAuth` — returns 401 if no session.
- `requireRole(...roles)` — returns 403 if the session role doesn't match.

The roles are **Super Admin**, **Dispatcher**, **Driver**, **Investor**. Routes either guard with `requireRole('Super Admin')` or accept multiple roles. The `requireAuth`-only routes (no role check) are the ones any logged-in user can hit.

Two additional gates:

- **Webhook secret.** n8n's load-ingestion endpoints require an `x-webhook-secret` header matching `N8N_WEBHOOK_SECRET`. There is no user session involved.
- **Demo viewer lockdown.** A global write-lockdown middleware blocks all mutating verbs (POST/PUT/PATCH/DELETE) for a demo-only role, with a small whitelist for harmless idempotent operations.

Role-based data sanitization

Two scrubbing points where data flows out:

- `GET /api/data` strips columns whose headers match `/rate|amount|revenue|pay|charge|price|cost/i` when the requester is a Driver.
- `PUT /api/data/:rowIndex` preserves broker and phone-contact columns by reading them back from the sheet and splicing them into the write payload, so a non-Super-Admin update cannot accidentally clobber broker info.

The pattern is "regex-detect sensitive columns at request time" — not "schema-declared sensitivity." This makes the system tolerant of differently-named columns across sheets but fragile to typos in column headers.

Error handling

Every route handler is wrapped in `try/catch` with a generic `500` JSON response on failure. There is no structured error type and no error-aggregation service — errors are `console.error`'d with enough context to diagnose from `pm2 logs`.

Multi-step operations are not atomic. If a dispatch assignment writes to the sheet successfully but the audit-trail insert fails afterward, the sheet has the new state and the audit table doesn't. The system tolerates this because:

- Audit failures don't block the user.
- Sheets-vs-SQLite drift is rare in practice and visible from the affected screen.

The geofence handler inside `POST /api/location` is the canonical example: location persistence is the primary obligation, and geofence-triggered status updates are best-effort — any error inside the geofence block is caught and logged but never causes the location report to fail.

Rate limiting

Per-endpoint limiters using `express-rate-limit`, naming pattern `{feature}Limiter`:

LIMITER	WINDOW	MAX	PURPOSE
`publicFormLimiter`	15 min	10	Public driver/investor application submission
`loginLimiter`	15 min	20	Brute-force protection on `/api/auth/login`
`changePasswordLimiter`	15 min	5	Slow password rotation attacks
`driverFilesLimiter`	15 min	30	Enumeration protection on `/api/trucks/:id/driver-files`
`truckDocViewLimiter`	15 min	30	Same for `/api/driver/truck-documents/:id/view`
`trackPublicLimiter`	15 min	60	Higher cap because customers refresh the public tracker often
`expenseOcrLimiter`	15 min	20	Caps Gemini API spend on receipt OCR

All limiters use `standardHeaders: true` so clients can read `X-RateLimit-*` headers. The 60-second in-memory Sheets cache is the other practical throttle — it absorbs bursty dashboard traffic before it ever reaches the limiter.

Real-time fan-out

Socket.IO is the way the UI stays current without polling. Clients emit `register` with their name (or role) to join a room. Server emits include:

- `load-assigned` — to a specific driver when dispatched.
- `load-cancelled` — to a driver when their dispatch is cancelled.
- `load-deleted` — to the dispatch room when a load is soft-deleted.
- `load-accepted` / `load-declined` — to the dispatch room on driver response.
- `status-updated` — to the dispatch room on every status transition.
- `dispatch-notification` — for everything that needs to surface in the admin notifications panel. Metadata always contains `loadId` so clicks can deep-link to `/dashboard?load=…`.
- `new-message` — to all on chat message.
- `pod-uploaded` — to the dispatch room when a driver uploads proof of delivery.
- `location-update` — to the dispatch room when a driver reports GPS.
- `geofence-trigger` — to driver and dispatch room when the geofence auto-advances a status.

- `reload` — to all clients 500 ms after server start, used as a dev-only live-reload signal.

Investors join an `investor` room — separate from the `dispatch` room — so dispatch traffic doesn't leak to them. Drivers join a per-driver room `driver:<name>` keyed by their full name (case-insensitive after normalization).

The load-exclusion helper

Three financial aggregators (`/api/dashboard`, `/api/financials`, `/api/investor`) all answer the question "is this load live?" the same way: they run sheet rows through `excludeDroppedLoads(rows, headers)` before any math. The helper drops:

- Rows whose status matches `CANCELED_STATUS_RE = /^(cancel|canceled|cancelled)$/i`.
- Rows whose Load ID is in the `deleted_loads` SQLite table.

This is the single chokepoint that keeps dashboard, financials, and investor numbers aligned. If you add a fourth aggregator, route it through the same helper or your number will drift from everyone else's.

Soft-delete patterns

Two patterns coexist by design:

Separate `deleted_loads` table. Used when the canonical record lives in Google Sheets, not SQLite — the soft-delete state is what lives locally. Query pattern: `LEFT JOIN deleted_loads … WHERE deleted_loads.load_id IS NULL`. Recovery: `DELETE FROM deleted_loads WHERE load_id = ?`.

`deleted_at` timestamp column. Used on tables that live entirely in SQLite (e.g., `job_applications`). List endpoints filter `WHERE deleted_at IS NULL`, with `?include_deleted=true` to recover. Cheaper than the join but only works when SQLite owns the record.

Pick the pattern that matches your table's home, not the other way around.

Frontend

The frontend is a Vue 3 single-page application built with Vite and served by Express in production from `client/dist/`. In development, Vite runs its own dev server on port 5173 and proxies `/api` and `/socket.io` to the Express server on port 3000.

Stack

- **Vue 3** with the Composition API throughout. No mixins, no class components.
- **Vite** for build and dev.
- **Vue Router** with role-based guards.
- **Pinia** for state. Each store is a separate module.
- **Tailwind CSS v4** for utility styling.
- **shadcn-vue** (via radix-vue / reka-ui) for primitive components: buttons, dialogs, tabs, selects, tables.
- **Vant** for mobile UI on driver and public surfaces. Reserved for those — admin views stick with shadcn-vue + Tailwind to keep desktop UX consistent.
- **Leaflet** and **Google Maps** for maps. Leaflet for the customer-facing tracker, Google Maps for everything that needs routing and geocoding.
- **Socket.IO client** for real-time updates.

Directory layout

```
client/
├── src/
│   ├── assets/         # global CSS (shared.css)
│   ├── components/     # feature-organized
│   │   ├── ui/         # shadcn-vue primitives
│   │   ├── layout/     # AppSidebar, AppShell
│   │   ├── shared/     # cross-feature widgets
│   │   ├── dashboard/  # JobBoardTab, ActiveLoadsTab, etc.
│   │   ├── driver/     # driver-specific tabs and modals
│   │   ├── investor/   # earnings, fleet breakdown, etc.
│   │   ├── invest/     # public investor application wizard
│   │   ├── apply/      # public driver application form
│   │   ├── trucks/, trailers/, users/, drivers-db/, investors/, data-manager/
│   ├── composables/    # useApi, useSocket, useToast, useGeolocation, …
│   ├── stores/         # Pinia stores
│   ├── views/          # 25 route-level views
│   ├── wizard/         # JSON-driven wizard engine (used by /invest)
│   ├── router/         # route definitions + guards
│   └── main.js
└── vite.config.js      # /api and /socket.io proxy → :3000
```

Routing

There are 25 routes. Each carries a `meta.roles` array and an optional `meta.alwaysPublic` flag.

ROUTE	ACCESS	NOTES
`/login`	public	Redirects authenticated users to their role home
`/apply`	public	Driver application form (no sidebar)
`/invest`	public	Investor application wizard
`/track`	public (`alwaysPublic`)	Customer search form
`/track/:loadId`	public (`alwaysPublic`)	Customer tracker — accessible to admins for preview
`/dashboard`	Super Admin, Dispatcher	
`/jobs/new`	Super Admin	
`/tracking`	Super Admin, Dispatcher	Live truck map
`/expenses`	Super Admin, Dispatcher	
`/invoices`	Super Admin	
`/messages`	Super Admin, Dispatcher	
`/notifications`	Super Admin, Dispatcher	
`/data`	Super Admin	Sheet data manager
`/driver`	Driver, Super Admin	No sidebar (mobile shell)
`/investor`	Super Admin, Investor	
`/users`	Super Admin	
`/trucks`	Super Admin, Dispatcher, Investor	
`/investors`	Super Admin	Investor records
`/drivers`	Super Admin	Drivers directory
`/trailers`	Super Admin, Dispatcher	
`/applications`	Super Admin	Driver applications

ROUTE	ACCESS	NOTES
`/investor-applications`	Super Admin	Investor applications
`/admin/tools`	Super Admin	
`/admin/financials`	Super Admin	Company P&L
`/archive`	Super Admin	Archived sheet data

The router runs `checkSession()` on the very first navigation and blocks until the session check resolves. Subsequent navigations use the cached `isAuthenticated` state. Routes with `meta.alwaysPublic` bypass the "authenticated → redirect to role home" rule, which is what lets logged-in dispatchers preview the public tracker without being punted back to their dashboard.

Unauthenticated users hitting a protected route get bounced to `/login`. Authenticated users hitting a route their role can't access get redirected to `auth.roleHome` (Driver → `/driver`, Dispatcher → `/dashboard`, Investor → `/investor`, Super Admin → `/dashboard`).

Composables

Several composables behave as module-level singletons rather than per-component instances. Important to understand:

- **`useApi()`** — returns the same axios-like fetch wrapper instance every time it's called. Each Pinia store does `const api = useApi()` at module scope.
- **`useSocket()`** — maintains a single global Socket.IO connection for the whole app. Components subscribe to events; teardown happens on unmount but the connection persists.
- **`useToast()`** — single global toast queue.

Other composables are per-instance:

- **`useGeolocation()`** — GPS tracking. Uses `watchPosition` with a 60 s stationary heartbeat, 50 m movement threshold, and 10 s minimum inter-report interval. Reports to `POST /api/location` with the current load ID. Errors are silently swallowed so a GPS failure never disrupts the driver. Exposes a separate `requestPermission()` helper used by `DriverView` to gate the app on mount — fully-onboarded drivers see a "Location Access Required" lock until the browser returns `granted`.

- `useGoogleMaps()` — loads the Maps JS API via `@googlemaps/js-api-loader`. Fetches the API key from `GET /api/config/maps-key` so it doesn't have to ship to the client at build time.
- `useGeocode()` — wrapper over the geocode-cache endpoints.
- `usePagination()` — generic offset/limit pagination state.
- `useSocketRefresh()` — used by 11 admin pages to auto-refresh their data when a relevant socket event fires.

State management

Pinia stores by domain:

- **auth** — session, role, roleHome.
- **dashboard** — KPIs, job board, active loads, fleet data.
- **sheets** — generic sheet read/write.
- **driver** — driver-specific load list and status.
- **messages** — chat messages, optimistic appends.
- **investor** — investor dashboard data.
- **users**, **adminTools**, **dispatchNotifications**, **driversDb**, **investors**, **trucks**, **trailers**, **invoices**, **financials**.

Two stores apply optimistic updates: `driver` and `messages` both append locally before the API request completes, then reconcile on the response.

The app intentionally **does not** maintain a deep client-side cache. Most pages re-fetch on mount and rely on Socket.IO refresh events to stay current. This keeps the data model simple and the dashboard accurate.

The app shell

A shared `appShell` Pinia store exposes `isMobile` (driven by a resize listener) and `sidebarOpen`. On mobile the `AppSidebar.vue` renders as a slide-in drawer with a backdrop overlay; on desktop it's a persistent collapsible sidebar. Any new admin view should toggle the drawer via `appShell.openSidebar()` rather than rolling its own mobile nav.

Admin pages (Dashboard, Notifications, Messages, Expenses) are responsive top-down — Phase 0–4 work collapsed multi-pane layouts into single-pane stacks below Tailwind's `md` breakpoint and swapped detail tables for card lists.

The driver view

`/driver` is the largest single view in the app (`DriverView.vue`, ~1,700 lines). It runs without a sidebar because drivers use it on phones in the cab. It's organized as five tabs:

1. **Loads** — assigned, accepted, in-progress.
2. **Status** — current load detail and status update form.
3. **Alerts** — notifications.
4. **Kit** — truck documents the admin has marked visible, plus onboarding docs.
5. **Messages** — chat with dispatch.

A sixth screen — the **onboarding lock** — blocks the entire view if the driver hasn't completed onboarding. The lock progresses through document signing, drug test scheduling, FMCSA Clearinghouse enrollment, and driver training.

The investor view

`/investor` is a stack of sections that compose the dashboard:

- **Hero header** — profile photo, name, "Performance overview", date-range picker, "Download Report" button.
- **Earnings section** — month selector, full P&L breakdown for the selected month, all-time totals strip.
- **Fleet breakdown** — per-truck cards with mileage, status, assigned driver.
- **Production** — miles driven, loads completed, utilization.
- **Cash flow** — revenue trend chart.
- **Asset section** — truck inventory and book value.
- **Documents portal** — signed legal docs, tax forms.
- **Chat** — direct messaging with admin.

The investor view consumes `/api/investor` which returns a pre-aggregated payload — no client-side joins.

The wizard engine

The public investor application at `/invest` is a multi-step wizard built on a small JSON-driven framework in `client/src/wizard/`:

- `engine/WizardEngine.js` — step interpreter.
- `expressionEvaluator.js` — evaluates `show-if` and `require-if` conditions without `eval()`.
- `spotlight.js` — guided highlight overlay.
- `data/` — step schemas (the actual content of the wizard).

When adding a new multi-step form, extend this engine rather than rolling a bespoke stepper. The driver application at `/apply` is a simpler form and predates the wizard, so it lives as a single Vue component.

Build & deploy

- Development: `npm run dev` (Express, port 3000) and `npm run dev:client` (Vite, port 5173) in separate terminals. Open `http://localhost:5173`.
- Production build: `npm run build:client` produces `client/dist/`.
- Production serve: `npm start` — Express checks if `client/dist/` exists and serves it, otherwise falls back to the legacy `public/` directory.

The SPA catch-all `app.get("*")` serves `index.html` for any unmatched route, which is what makes client-side routing work in production.

Integrations

LogisX leans on several external services. This chapter covers each integration, why it's there, how it's authenticated, and how to recover if it goes down.

Google Sheets — system of record for loads

The "Job Tracking" tab in spreadsheet `1ey1n0AAG0k8k-qwkWh2T_C8VqqY129OQQr7D5wNl7Mo` is where every active load lives. The spreadsheet is shared with the service account `sheets-bot@logistics-app-491014.iam.gserviceaccount.com` as Editor.

Auth. A JSON service-account key at the project root (`service-account-key.json`, not in git). The Google API client is lazy-initialized as a singleton via `getSheets()`.

Quota. 300 requests per minute per project. The 60-second in-memory cache (`getJobTrackingCached()`) is the practical throttle — without it, a dashboard refresh fan-out could blow the quota in seconds.

Write semantics. `valueInputOption: "USER_ENTERED"` so spreadsheet formulas in payloads are interpreted, not treated as literal strings. Rows are 1-indexed (row 1 = headers, row 2+ = data). DELETE converts to 0-indexed for the Sheets `batchUpdate` `deleteDimension` call.

Sheet ID caching. Tab GIDs are looked up once per process and cached in a `Map` to avoid repeated metadata API calls.

Recovery. If a row write fails partway (the network drops between the read and the write), the sheet may be left in an intermediate state. The status update endpoint guards against this by reading the entire sheet, verifying the Load ID at the given row index still matches, and falling back to a load-ID scan if not.

Google Drive — POD storage

Driver-uploaded proof-of-delivery photos are converted to PDF via `imageToPdf()` (pdfkit) and uploaded to a single shared folder identified by `GOOGLE_DRIVE_FOLDER_ID`. The returned Drive file ID is stored in `documents.drive_file_id` so we can render a "View on Drive" link later.

If `GOOGLE_DRIVE_FOLDER_ID` is unset, the upload silently skips Drive and stores only the local copy. This is the path used in development.

Scopes. `drive.file` only — the service account can read and write only files it created, not the whole Drive.

Google Maps — routing, geocoding, weather

Three Maps APIs are in use:

- **Directions** for live route calculation on `/api/route`.
- **Geocoding** for converting Pickup/Drop-off addresses into lat/lng. Results are cached forever in the `geocode_cache` SQLite table.
- **Places** for the address autocomplete in driver/investor application forms.

A small `/api/weather` proxy exists as well — though it uses a different provider, the API key is on the same shared key store.

Auth. A single API key in `GOOGLE_MAPS_API_KEY`. Exposed to the frontend via `GET /api/config/maps-key` so it doesn't need to ship at build time. The key should be restricted to LogisX's domains in Google Cloud Console (this is currently parked until Cloud Console access is provisioned).

Geocode cache. Hits return immediately; misses call the Maps API and persist the result. There is no TTL — addresses don't change. To force a re-geocode, delete the row from `geocode_cache`.

Google Gemini 2.5 Flash — receipt OCR

Drivers can photograph a fuel or repair receipt and the form pre-fills from a Gemini vision call. The integration lives at `POST /api/expenses/ocr`.

Why Gemini and not Tesseract. Tesseract was the first attempt. Accuracy on phone-camera receipts was too low — wrong totals, miscoded vendors, missing odometer readings. Gemini 2.5 Flash, called with a `responseSchema` so the API itself enforces the JSON shape we expect, dramatically outperforms Tesseract on this task.

Auth. `GEMINI_API_KEY` in `.env`. Called via `fetch` (no SDK), with a 15-second `AbortController` timeout and two retries on backoff — same pattern as the Routemate client.

Output. A structured `{amount, date, vendor, gallons, odometer, suggestedType, confidence}` object. The driver's form prefills these fields, the driver confirms or edits, and only then is the expense saved.

Cost gate. `expenseOcrLimiter` caps each user to 20 OCR calls per 15 minutes.

Graceful degradation. When `GEMINI_API_KEY` is unset, the endpoint returns 503 and the form silently falls back to manual entry. The user experience is "OCR was unavailable" rather than "the app broke."

n8n — broker email ingestion

When a broker emails a Rate Confirmation to `info@logisx.com`, an n8n workflow parses the PDF and posts a structured payload to `POST /api/n8n/job`. The server appends a new row to the Job Tracking sheet with status `Unassigned`, and a `dispatch-notification` socket event fires immediately so dispatchers see the new load without refreshing.

Auth. Shared secret in the `x-webhook-secret` header. Matches `N8N_WEBHOOK_SECRET` from `.env`. n8n stores the secret in its own credentials.

Endpoints.

- `POST /api/n8n/job` — primary ingestion path.
- `POST /api/webhook/new-load` — secondary, used for re-pushing after manual fixes in n8n.

Replay. A `replay-via-webhook-injection.js` script (committed alongside the n8n workflows) lets us re-run a single message through the pipeline when a workflow change should be applied retroactively.

PDF parsing. The actual PDF parsing happens inside n8n using LlamaParse, with a "Validate Parse Quality" node that gates on confidence before posting downstream. The server doesn't see the PDF — it only receives the structured payload.

Routemate — ELD telematics (off by default)

Routemate is FMCSA-certified ELD hardware in each truck. LogisX pulls live GPS, fuel diagnostics, fault codes, and DVIR reports from their cloud REST API.

Adapter. A single point of contact at `lib/routemate-client.js`. Every server-side caller goes through this adapter, never direct. The adapter handles auth (`X-Api-Key` header), retry/backoff, and a 15-second `AbortController` timeout — same template as the Gemini client.

Master switch. `ROUTEMATE_ENABLED` in `.env`. **Defaults to off.** All sync intervals are dormant and the manual probe returns 503 until this is flipped on.

Pull cadence. Three intervals, each configurable:

- `ROUTEMATE_POLL_LIVE_SEC` (60) — live telemetry sync.
- `ROUTEMATE_POLL_FAULTS_SEC` (300) — fault-code sync.
- `ROUTEMATE_POLL_DAILY_HOUR` (4) — daily rollups (MPG, HOS).

Tables. Six SQLite tables, all `IF NOT EXISTS` (additive, reversible):

- `routemate_vehicles` — local mirror of the Routemate vehicle inventory.
- `routemate_telemetry` — append-only GPS feed.
- `routemate_fault_codes` — one row per active code per vehicle.
- `routemate_dvir` — DVIR inspection reports.
- `routemate_fuel_daily` — telemetry-derived MPG rollup.
- `routemate_hos_daily` — driver duty-time rollup.

Linking trucks. Each truck row in the `trucks` table has a `routemate_vehicle_id` column. Admins set this in the Trucks UI to link a LogisX truck to a Routemate vehicle.

Live GPS source priority. `GET /api/locations/latest` prefers a Routemate telemetry row younger than 5 minutes over the phone-based `driver_locations` row. The response is tagged with `source: 'routemate' | 'phone'` so consumers can show provenance. Old consumers that ignore `source` continue to work because the response shape is otherwise unchanged.

Phone GPS as fallback. `useGeolocation.js` and `POST /api/location` are untouched by Routemate — they remain the fallback path. If Routemate goes down, phone GPS takes over with no code change.

Tesseract — legacy POD OCR

Tesseract still runs on POD image uploads only (`POST /api/documents/upload`). It exists in case the POD photo contains a stamp or signature we want to capture as text. Errors are silently swallowed — OCR failure never blocks the underlying upload.

Tesseract was previously used for expense receipts as well, but was unwired once accuracy turned out to be too low. That path now goes to Gemini.

Nodemailer — outbound email

Used by:

- Driver onboarding acceptance emails.
- Investor outreach campaigns (`/api/investor-outreach/send`).

Auth. `GMAIL_USER` and `GMAIL_APP_PASSWORD` in `.env`. Standard Gmail app password — the account itself is `info@logisx.com`.

Putting it together — a load's lifecycle

```
Broker email → n8n parses PDF → POST /api/n8n/job (x-webhook-secret)
    └→ Sheet row appended (status: Unassigned)
            └→ Socket "dispatch-notification" → admin notifications panel
                    └→ Dispatcher assigns driver in /dashboard
                            └→ Socket "load-assigned" → driver phone
                                    └→ Driver taps Accept
                                            └→ Driver phone GPS → POST /api/location
                                                    └→ (Routemate, if enabled, also
reporting)
                                                            └→ Geofence enters pickup
→ status auto-advances
                                                                    └→ Driver uploads
POD → Drive upload
                                                                            └→

Status: Delivered

└→ Super Admin generates invoice (pdf-lib/pdfkit)

└→ Approve → payment
```

Every external system in that chain is replaceable: swap the email parser, change the GPS source, move the POD store. The integration boundaries are narrow enough (a webhook, an API call, a file upload) that this is mostly a one-file edit per replacement.

PDF Generation & Document Templates

LogisX generates a lot of PDFs: driver employment applications, weekly invoices, signed onboarding documents (driver and investor), investor reports, master agreements, vehicle leases, W-9 forms, and the documentation you are reading right now. Three libraries cover the field, each chosen for a different reason.

Three libraries, three jobs

LIBRARY	WHEN TO USE IT	EXAMPLE
pdfkit	Programmatic PDFs built from scratch — write text, draw boxes, embed images. Best for one-off documents with a fixed structure.	Employment application PDF, image-to-PDF conversion for POD photos.
pdf-lib	Manipulating existing PDFs — merging multiple files, embedding signatures into a pre-built form, filling AcroForm fields.	Merging the application PDF with uploaded certificates; filling W-9 AcroForm fields.
Puppeteer (via `lib/pdf-browser.js`)	Rendering HTML/CSS to PDF. Best when the document needs designer-grade layout, fonts, and styling.	Onboarding policies, weekly service invoices, master agreement, vehicle lease, investor reports, this documentation.

When in doubt, use Puppeteer. HTML is the most maintainable substrate — a designer can edit a template without touching code.

The Puppeteer pipeline

`lib/pdf-browser.js` is the canonical entry point. Its API:

- `getBrowser()` — returns a singleton Chromium instance. If Chromium crashes (OOM, SIGKILL), the next call relaunches transparently. A mutex serializes concurrent launches so two requests don't race during a relaunch.

- **`renderHtmlToPdf(html)`** — takes a raw HTML string and returns a `Buffer`. Internally:
  1. Open a new page.
  2. `setContent(html, { waitUntil: "networkidle0", timeout: 30000 })`.
  3. `await page.evaluateHandle(() ⇒ document.fonts.ready)` — critical, without it the first render uses Arial fallback because Google Fonts haven't finished loading.
  4. `page.pdf({ format: "Letter", printBackground: true, margin: 0, preferCSSPageSize: true })` — margins are delegated to the template's `@page { margin: 1in }` so the template is the single source of truth.
  5. Wrap in `Buffer.from(pdf)` because Puppeteer ≥22 returns `Uint8Array`, which Express's `res.send()` mis-serializes to JSON without the wrap.
- **`shutdownBrowser()`** — called on process termination to close the singleton.

This module is the only place in the codebase that talks to Puppeteer. Every other PDF-via-HTML caller (invoices, onboarding documents, this documentation) goes through `renderHtmlToPdf()`.

Policy templates

Onboarding documents — equipment policy, mobile policy, substance policy, contractor agreement, investor master agreement, vehicle lease — live as HTML templates in `onboarding-templates/policy/`. Each template is a plain HTML page with inline CSS, Google Fonts loaded by `<link>`, and form inputs using `aria-label` attributes as placeholders:

```
<input aria-label="Full name" />
<input aria-label="Date" />
```

The corresponding field map in `lib/policy-field-maps.js` declares which `aria-label` matches which data field. The renderer in `lib/policy-renderer.js` walks the field map and uses Puppeteer's `page.evaluate()` to populate inputs by `aria-label` lookup at render time.

Why `aria-label` and not `{{mustache}}`. Two reasons. First, `aria-label` is real HTML — the template still renders as a fillable form in a browser, which is useful for proofing. Second, DOM-based population can do things a string-replace template can't: replace specific inputs with `<img>` tags for hand-drawn signatures, clone table rows for multi-vehicle Exhibit A documents, conditionally check checkboxes by ID or label.

Multi-vehicle Exhibit A. The Master Participation Agreement template has a single Exhibit A row. When an investor owns multiple trucks, the renderer clones that row N times before rendering. This is one of the things string-replace templating cannot do, and is the reason `policy-renderer.js` exists.

Owner-operator weekly invoice. Same pattern — the template has placeholder rows for loads and expenses; the renderer clones them based on the actual week's data.

The policy field map

`policy-field-maps.js` exports a mapper per document type. Each mapper is a function that takes a data object and returns:

```
{
  text: { /* aria-label → text value */ },
  checkboxesById: { /* element id → boolean */ },
  checkboxesByLabel: { /* aria-label → boolean */ },
  signatureImage: '<base64 PNG>',
  signatureLabelForImage: 'Driver signature',
  vehicles: [ /* array, drives Exhibit A cloning */ ],
  loadsRows: [ /* for owner-op invoice */ ],
  expensesRows: [ /* for owner-op invoice */ ],
  dayCompletedByIndex: { /* day index → "completed" | "n-a" */ },
}
```

The renderer treats every key as optional. Add a new field to a template by adding an `<input aria-label="…">`, then adding the matching entry to the mapper.

Where each template is used

TEMPLATE FILE	ENDPOINT	WHEN IT RENDERS
`equipment-policy.html`	`GET /api/onboarding/documents/equipment_policy/pdf`	Driver onboarding (3 simple text fields).
`mobile-policy.html`	Same pattern	Driver onboarding (~5 fields).
`substance-policy.html`	Same pattern	Driver onboarding.
`contractor-agreement.html`	Same pattern	Driver onboarding (heaviest — ~25 fields + banking info + payment method checkboxes).
`service-invoice.html`	`POST /api/invoices/generate`	Weekly driver invoice (fixed-rate driver pay).
`service-invoice-owner-op.html`	Same endpoint, different `pay_type`	Weekly invoice for owner-operators (percentage of revenue, with cloneable loads/expenses tables).
`master-agreement.html`	`GET /api/public/investor-onboarding/:id/documents/master_agreement/pdf`	Investor onboarding (~25 fields, equipment checkboxes, multi-vehicle Exhibit A).

TEMPLATE FILE	ENDPOINT	WHEN IT RENDERS
`vehicle-lease.html`	Same pattern	Investor onboarding (lessee/lessor split, 10-item inspection checklist).

W-9 — a different mechanism

The W-9 is the one onboarding document that does **not** use the HTML-template pipeline. It uses pdf-lib to fill AcroForm fields on a pre-built PDF (`docs/w9-test.pdf`-style template). The `fillW9Form()` helper in `server.js` looks up each AcroForm field by name and sets its value. The result is then merged into the investor's signed-document bundle alongside the HTML-rendered docs.

Why not use HTML? The W-9 is a federally-defined form with exact layout requirements. Reproducing it in HTML would be more work and harder to audit; the IRS already publishes a fillable PDF.

Imageto-PDF conversion

POD photos uploaded by drivers are not stored as raw images on Drive — they are wrapped in a single-page PDF via `imageToPdf()`. This keeps the document set homogenous (everything in the Drive folder is a PDF) and lets us add a header/footer with metadata if we want to.

How this documentation is generated

The PDF you are reading is built by `scripts/docs/generate-docs.js`, which:

1. Reads markdown files from `docs/manual/{technical|user}/` in sort order.
2. Concatenates them and runs `marked.lexer()` to extract headings for the auto-generated table of contents.
3. Renders each markdown file with `marked.parse()`, wraps each in a `<section class="chapter">` element (CSS `page-break-before: always`), and stitches them together.
4. Wraps in an HTML shell that loads Google Fonts and the shared `docs/manual/assets/styles.css`.

5. Rewrites all `<img>` `src` attributes to inline `data:` URIs so Puppeteer doesn't need network access to resolve screenshots.
6. Calls `renderHtmlToPdf()` from `lib/pdf-browser.js` — the same path every other production PDF goes through.

Total dependencies for the documentation pipeline: `marked` (devDependency). Everything else — Puppeteer, the singleton browser, the font-loading dance — is already there.

Adding a new policy document

The full workflow:

1. **Create the HTML template.** Drop `onboarding-templates/policy/my-doc.html` with `aria-label` inputs where data should go. Use inline CSS and load Google Fonts via `<link>` for visual parity with the existing documents.
2. **Add a field map.** In `policy-field-maps.js`, add `myDoc(data) { return { text: {...}, checkboxesById: {...} } }`.
3. **Wire up the endpoint.** In `server.js`, add a route that calls `renderPolicy('my_doc', data)` from `policy-renderer.js`.
4. **Test.** Hit the endpoint, open the resulting PDF, verify every field landed in the right spot.

No new dependencies, no new abstractions. The pattern scales linearly with the number of document types.

Deployment

LogisX runs on a single VPS at `76.13.22.110` under `/var/www/logistics-app`, managed by `pm2` with the process name `logistics-app`. A staging process named `logisx-staging` runs on the same VPS for pre-production testing.

Production runtime

- **Host:** Single VPS, Ubuntu, public IP `76.13.22.110`.
- **App path:** `/var/www/logistics-app`.
- **Process manager:** `pm2`. The app process is `logistics-app`; the staging process is `logisx-staging`.
- **Web server in front:** nginx, terminating TLS on `app.logisx.com` and reverse-proxying to the local Node process.
- **Domain split:** `app.logisx.com` is the Express app. `logisx.com` is a separate Wix marketing site, not part of this codebase.

Environment configuration

The production environment requires these files at the project root:

- `.env` — environment variables (see below).
- `service-account-key.json` — Google service account credentials.

Required `.env` variables

```
SESSION_SECRET=<long random string>           # required in production
GOOGLE_DRIVE_FOLDER_ID=<Drive folder ID>      # optional, POD uploads skip Drive if
empty
GOOGLE_MAPS_API_KEY=<Maps API key>            # required for maps, routing, geocoding
GMAIL_USER=<info@logisx.com>                  # for onboarding/outreach email
GMAIL_APP_PASSWORD=<Gmail app password>       # nodemailer auth
GEMINI_API_KEY=<Gemini API key>               # optional, enables receipt OCR
N8N_WEBHOOK_SECRET=<shared with n8n>          # required for n8n ingestion
PORT=3000                                     # optional, defaults to 3000
NODE_ENV=production
```

Optional `.env` variables

```
GEMINI_OCR_MODEL=<override default model>     # defaults to gemini-2.5-flash
ROUTEMATE_BASE_URL=https://cloud.routemate.ai
ROUTEMATE_API_KEY=<Routemate key>
ROUTEMATE_ENABLED=false                       # default off until key is wired in prod
ROUTEMATE_POLL_LIVE_SEC=60
ROUTEMATE_POLL_FAULTS_SEC=300
ROUTEMATE_POLL_DAILY_HOUR=4
```

`service-account-key.json`

A standard Google Cloud service account JSON key for the account `sheets-bot@logistics-app-491014.iam.gserviceaccount.com`. Both the production spreadsheet and the Drive POD folder must be shared with this email as **Editor**.

Deploy workflow

The standard flow after merging to `main`:

```
ssh root@76.13.22.110 "cd /var/www/logistics-app && \
  git pull --ff-only origin main && \
  npm install --silent --no-audit --no-fund && \
  npm run build:client --silent && \
  pm2 restart logistics-app --silent"
```

This is intentionally one shell pipeline, not a deploy script. It runs in this order:

1. Pull main, fast-forward only — no merge commits, no rebase, fail if not fast-forward.
2. Install npm dependencies. The `postinstall` hook on the root `package.json` also installs `client/`'s dependencies.
3. Build the Vue client to `client/dist/`.
4. Restart `pm2`. The `--silent` flag suppresses chatter — failures still surface via exit code.

If any step fails, the deploy aborts mid-flight and the previous build keeps serving — `pm2` only restarts the process if the script reaches step 4.

nginx configuration

nginx is the public-facing server. Two adjustments matter for LogisX:

- **`client_max_body_size 50m`** — necessary because the Express body limit is 50 MB to accept large base64 payloads with embedded photos and signatures. Without this, multipart uploads from drivers (large POD photos) get 413'd at the nginx layer.
- **WebSocket upgrade headers** — `proxy_set_header Upgrade $http_upgrade` and `proxy_set_header Connection "upgrade"` so Socket.IO can negotiate from polling to WebSocket transport.

TLS certificates are managed by Let's Encrypt. Renewal runs from cron.

pm2 configuration

The `logistics-app` process is started with `pm2 start server.js --name logistics-app`. Useful pm2 commands:

- `pm2 logs logistics-app` — tail stdout/stderr.
- `pm2 logs logistics-app --lines 200` — recent logs.

- `pm2 restart logistics-app` — restart in-place. Brief downtime (seconds) while Node restarts.
- `pm2 reload logistics-app` — zero-downtime reload (only useful when the app supports it; for a single-process app like ours, `restart` is equivalent in practice).
- `pm2 monit` — live CPU/memory dashboard.
- `pm2 save` — persist the process list across reboots.
- `pm2 startup` — register pm2 with systemd so it survives a host reboot.

The staging process (`logisx-staging`) is the same `server.js` against a separate database file and port. Useful for testing a deploy before promoting to production.

Build vs. dev

Three commands cover the matrix:

COMMAND	WHAT IT DOES	WHEN
`npm start`	`node server.js`	Production / staging.
`npm run dev`	`nodemon --ext js server.js`	Local backend development with auto-restart on file changes.
`npm run dev:client`	Vite dev server on port 5173, proxying `/api` and `/socket.io` to :3000	Local frontend development.
`npm run build:client`	Vite production build to `client/dist/`	Pre-deploy step.

In development, run `npm run dev` and `npm run dev:client` in two terminals and open `http://localhost:5173`. In production, only `npm start` runs — Express serves the pre-built `client/dist/`.

What lives on the VPS but not in git

- `.env` — environment secrets.
- `service-account-key.json` — Google credentials.
- `app.db` — SQLite database. Backed up via the manual download endpoint.
- `uploads/` — local copies of POD photos, receipts, signatures. (Drive holds the canonical PODs.)

Nothing else. The repo is the source of truth for code; the VPS holds runtime state.

Hardening

Several defaults that protect the production environment:

- `SESSION_SECRET` is enforced — the server warns at boot if it falls back to the dev default. In production, the warning becomes a real risk.
- **Cookie flags** — `httpOnly`, `secure: true` when `NODE_ENV=production`, `sameSite: 'strict'`.
- `compression` middleware gzips all responses.
- `express-rate-limit` caps the abusable endpoints (see backend chapter).
- **Demo viewer write lockdown** — a global middleware blocks all mutating verbs (POST/PUT/PATCH/DELETE) for a special demo role with a small whitelist for harmless idempotent operations.

Rollback

If a deploy goes wrong:

```
ssh root@76.13.22.110
cd /var/www/logistics-app
git log --oneline -10                    # find the last good commit
git checkout <good-commit>               # detach HEAD to that commit
npm install --silent --no-audit --no-fund
npm run build:client --silent
pm2 restart logistics-app --silent
```

The `app.db` SQLite file is not touched by a code rollback — schema migrations are additive, so older code reads a newer schema fine. The exception is the rename-recreate migration: if you deployed code that triggered a CHECK-constraint table recreate, a rollback to older code that doesn't know about the new constraint may or may not work depending on the constraint. When in doubt, restore `app.db` from a backup.

To restore main:

```
git checkout main
git pull --ff-only origin main
# … rebuild and restart as in the deploy flow above
```

Backups

There is no automated backup currently. The SQLite database can be downloaded by a Super Admin via `GET /api/db/download` — wired up for exactly this purpose. The Google Sheet is itself the system of record for loads and is implicitly backed up by Google.

Plan ahead for a routine snapshot of `app.db`, `.env`, `service-account-key.json`, and the contents of `uploads/`. The first three are tiny; `uploads/` can be larger depending on POD/receipt volume.

Two-process model with staging

Both `logistics-app` (production) and `logisx-staging` (staging) run on the same VPS. They share the host but have separate `.env`, separate SQLite databases, and separate ports. This is enough isolation for "test a deploy before promoting" — not enough for "run integration tests against a fresh DB." For destructive testing, point staging at a seeded copy of `app.db` (via `scripts/truncate-and-seed.js`) before exercising it.

Operations

Day-to-day operations: monitoring, scripts, audit, backups, common incident playbooks.

Audit trail

Admin-significant mutations write a row to the `audit_trail` SQLite table. The schema:

COLUMN	TYPE	NOTES
`id`	INTEGER PRIMARY KEY	
`action`	TEXT	A short slug — `user_created`, `driver_renamed`, `routemate_sync`, etc.
`user_id`	INTEGER	The acting user.
`target`	TEXT	What was acted on — usually a load ID, user ID, or "all".
`details`	TEXT	JSON blob with the before/after where applicable.
`created_at`	TIMESTAMP	UTC.

Currently logged actions include user CRUD, driver renames (which fan out to multiple sheets), Routemate sync runs, and several `scan-*` admin tools. The action vocabulary grows organically — add entries when you add a new admin tool. There is no scheduled retention; rows accumulate forever.

`/api/admin/audit-trail` exposes this to Super Admins as a paginated list.

Helper scripts

Live in `scripts/`:

SCRIPT	PURPOSE
`reset-super-admin-password.js`	Reset the Super Admin password against the local SQLite DB.
`truncate-and-seed.js`	Wipe and reseed `app.db` for clean-slate testing.
`seed-staging.js`	Seed a staging DB.
`create-demo-user.js`	Create a demo user for testing.
`geocode-loads.js`	Backfill geocodes for rows in "Job Tracking".
`generate-timeline-docx.py`	One-off Python script to render `.docx` session timelines (requires `python-docx`).
`docs/generate-docs.js`	Build the technical and user PDFs.
`docs/capture-screenshots.js`	Capture the UI screenshots used by the user manual.

Run scripts from the project root: `node scripts/<name>.js`. Each script is independent — no shared CLI framework, no shared config.

Monitoring & log access

There is no APM or log-aggregation service. The two practical mechanisms are:

- `pm2 logs logistics-app` — live tail of stdout and stderr. Errors land here.
- `audit_trail` **table** — every admin-significant mutation. Run ad-hoc SQL via `GET /api/db/query/audit_trail` (Super Admin only).

Errors that surface to users return generic 500 JSON; the diagnostic detail is in pm2 logs. When investigating a user-reported issue, ask for the approximate time and look at `pm2 logs --lines 500` from that window.

Backups

The SQLite database is the only mutable state the repo doesn't own. Take a snapshot of:

- `app.db` — via `GET /api/db/download` (Super Admin) or `scp` directly from the VPS.
- `.env` — runtime secrets.

- `service-account-key.json` — Google credentials.
- `uploads/` — local copies of POD photos, signatures, receipts.

Backups should be off-host. Cadence depends on tolerance for data loss; nightly is reasonable for the database, weekly is fine for everything else.

The Google Sheet is implicitly backed up by Google; you don't need to snapshot it yourself unless you want to be able to read it during a Google outage.

Common incident playbooks

Sheets API quota exhausted

Symptom. `/api/dashboard` returns 500. Logs show `Quota exceeded for quota metric 'Read requests'`.

Cause. 300 read requests per minute exceeded. Usually caused by a spike in dashboard refreshes when the in-memory cache hasn't warmed.

Mitigation. Wait — the limit refills every minute. Check that the cache (`getJobTrackingCached()`) is being used by the affected endpoint. If a new endpoint is bypassing the cache, route it through `getJobTrackingCached()` instead of calling Sheets directly.

Driver GPS not updating

Symptom. Tracking map shows a stale driver position.

Cause to check, in order:

1. **Browser permission.** The driver's browser blocks geolocation. `DriverView` should be showing the "Location Access Required" gate — confirm the driver sees it.
2. **Phone background.** iOS especially backgrounds the tab aggressively. The 60-second stationary heartbeat is the workaround; if a driver hasn't moved 50 m in 60 s, no report goes out. This is intentional but can look like "broken GPS" on the map.
3. **Server-side dedup.** `POST /api/location` has a 10-second minimum inter-report interval per driver. Reports faster than that are ignored.
4. **Routemate fallback path.** If `ROUTEMATE_ENABLED=true`, the GPS source for that driver may have switched to Routemate. The `source: 'routemate' | 'phone'` field on `/api/locations/latest` tells you which.

A load is stuck in the wrong status

Symptom. A driver completed delivery but the dashboard still shows "In Transit" or similar.

Cause to check.

1. **Duplicate Load ID rows.** The deduplicator keeps the bottom-most row per Load ID; if the corrected row is above the stale row, the stale row wins. Look at the sheet directly.
2. **Status guard.** Status updates enforce a "valid predecessor" rule. If the current status doesn't match the expected predecessor, the transition is rejected with a 409. Look at the response in the driver's browser console.
3. **Wrong-row write.** Older status updates didn't verify the row index against the Load ID before writing. The current code does, falling back to a Load ID scan if mismatch is detected. If a status update lands on the wrong row, it's a regression worth investigating.

Soft-deleted load reappeared

Symptom. A load that was deleted via `/api/loads/:loadId` shows up again in lists.

Cause. The Google Sheet row was edited (manually or by n8n) after the soft-delete, which generally doesn't restore the row but can if the row was on a Load ID that was previously in `deleted_loads` and the `deleted_loads` row got removed.

Mitigation. Re-run the soft-delete. Investigate why the `deleted_loads` row went away — possibly a manual cleanup.

Investor dashboard shows $0 earnings

Symptom. Investor logs in, sees zero across the board even though their truck has been running.

Cause to check.

1. **Carrier name mismatch.** The investor's `users.company_name` must match (case-insensitive) the Carrier Database carrier names assigned to their drivers. Spelling differences ("JONHSON" vs. "JOHNSON" — a real past bug) cause the join to miss.
2. **Owner ID column.** The Job Tracking sheet has an "Owner ID" column that links a load row to an investor. The investor view prefers Owner ID over the carrier-name fallback. If Owner ID is blank on every load, only the carrier-name fallback runs — which is fragile.
3. **Excluded loads.** The `excludeDroppedLoads` helper filters cancelled and soft-deleted loads from every aggregator. If every load was somehow flagged cancelled, the dashboard would zero out.

Webhook secret rejected

Symptom. n8n reports 401 from `POST /api/n8n/job`.

Cause. `N8N_WEBHOOK_SECRET` in the server `.env` doesn't match the value n8n is sending in `x-webhook-secret`.

Mitigation. Reset both sides to the same value. After rotating, run a single test message through n8n's manual replay and verify the row appears in the sheet.

Rate-limit telemetry

`express-rate-limit` sets `X-RateLimit-Limit`, `X-RateLimit-Remaining`, and `X-RateLimit-Reset` headers on every response. To diagnose "is this rate-limited?", inspect the response headers in the browser dev tools. The limiters are listed in the backend chapter.

Database admin tools

`GET /api/db/tables` lists every table. `GET /api/db/query/:table` returns rows from a single table with optional `?where=` and `?limit=`. `GET /api/db/download` returns the entire `app.db` file. All three are Super Admin only.

For ad-hoc SQL beyond what `/api/db/query` exposes, `scp` the database off the VPS and open it in DB Browser for SQLite or `sqlite3` CLI locally — never write SQL directly against the production file.

Schema scans

Three admin endpoints help find data-quality issues:

- `GET /api/admin/scan-duplicates` — duplicate rows in sheets.
- `GET /api/admin/scan-driver-mismatches` — driver name inconsistencies between sheet and Carrier Database.
- `GET /api/admin/scan-orphans` — orphaned data.
- `GET /api/admin/scan-stale-locations` — driver locations that haven't updated in too long.

Run periodically as a sanity check. Each surfaces issues that aren't usually visible in normal admin views.

When something is on fire

The right escalation order:

1. **Check** `pm2 logs` for the last 5 minutes of output.
2. **Check** `pm2 monit` for CPU/memory — Puppeteer can OOM the box if a runaway render queue forms.
3. `pm2 restart logistics-app` if the process looks wedged. Brief downtime.
4. **Roll back the last deploy** if the symptoms started right after one (see deployment chapter).
5. **Restore** `app.db` **from backup** as a last resort — there is no automated point-in-time recovery.

Most issues are recoverable without data loss because the Google Sheet holds the system of record for loads. SQLite tables (users, expenses, onboarding, telemetry) need their own backups.

API Reference

A flat list of every REST endpoint in `server.js`, grouped by domain. Each row shows the verb, path, and required role. See the relevant chapter for behavior.

Authentication

VERB	PATH	ROLE	PURPOSE
GET	`/api/auth/setup-check`	public	Is the system initialized?
POST	`/api/auth/setup`	public	One-time Super Admin creation.
POST	`/api/auth/login`	public	Establish session (rate-limited).
POST	`/api/auth/logout`	any	Destroy session.
GET	`/api/auth/session`	any	Current session info.

Users

VERB	PATH	ROLE	PURPOSE
GET	`/api/users`	Super Admin	List users.
POST	`/api/users`	Super Admin	Create user.
PUT	`/api/users/:id`	Super Admin	Update user.
DELETE	`/api/users/:id`	Super Admin	Delete user.
PUT	`/api/users/:id/rating`	Super Admin	Set rating.
GET	`/api/users/investors`	Super Admin	Users with Investor role.

Sheets CRUD

VERB	PATH	ROLE	PURPOSE
GET	`/api/tabs`	any	List sheet tab names.
GET	`/api/data?sheet=&page=&limit=`	any	Read rows (paginated).
POST	`/api/data?sheet=`	any	Append row.
PUT	`/api/data/:rowIndex?sheet=`	any	Update row.
DELETE	`/api/data/:rowIndex?sheet=`	Super Admin	Delete row.

Dashboard & dispatch

VERB	PATH	ROLE	PURPOSE
GET	`/api/dashboard`	Super Admin, Dispatcher	KPIs, job board, active loads, fleet.
POST	`/api/dispatch`	Super Admin, Dispatcher	Assign load to driver.
POST	`/api/dispatch/reassign`	Super Admin, Dispatcher	Reassign load.
POST	`/api/dispatch/cancel`	Super Admin	Cancel load (status → `Cancelled`).
DELETE	`/api/loads/:loadId`	Super Admin	Soft-delete via `deleted_loads`.
POST	`/api/driver/respond`	Driver	Accept/decline assignment.
GET	`/api/load/:loadId`	any	Single load detail.
PUT	`/api/load/:loadId`	any	Update load fields.

Driver

VERB	PATH	ROLE	PURPOSE
GET	`/api/driver/:driverName`	Driver, Super Admin	Driver-specific data (financials scrubbed).
PUT	`/api/driver/status`	Driver	Update load status.
GET	`/api/driver/truck-documents/:id/view`	Driver	View truck doc (rate-limited).
GET	`/api/driver/shared-documents/:id/download`	Driver	Admin-shared doc download.

Fleet

VERB	PATH	ROLE	PURPOSE
`/api/trucks`	CRUD	Super Admin (varies by op)	Trucks.
`/api/truck-assignments`	GET	any	Truck-driver assignments.
`/api/trailers`	CRUD	Super Admin, Dispatcher	Trailers.
`/api/drivers-directory`	CRUD	Super Admin	Drivers directory (SQLite Carrier Database).

Applications & onboarding

VERB	PATH	ROLE	PURPOSE
POST	`/api/public/apply`	public	Driver application (rate-limited).
GET	`/api/applications`	Super Admin	List applications.
PUT	`/api/applications/:id/status`	Super Admin	Approve/reject.
GET	`/api/applications/:id/pdf`	Super Admin	Download application PDF.
POST	`/api/public/investor-apply`	public	Investor application (rate-limited).
`/api/investor-applications`	CRUD	Super Admin	Manage.
`/api/public/investor-onboarding/:id/*`	public	Investor onboarding flow.	
`/api/onboarding/*`	Driver	Driver onboarding flow.	

Investors

VERB	PATH	ROLE	PURPOSE
`/api/investors`	CRUD	Super Admin	Investor records.
`/api/investor`	GET	Super Admin, Investor	Investor dashboard data.
`/api/investor/config`	GET/PUT	Super Admin, Investor	View configuration.
`/api/investor/documents`	GET	Super Admin, Investor	Signed documents.
`/api/investor/tax-csv`	GET	Super Admin, Investor	Tax CSV export.
`/api/investor/report`	GET	Super Admin, Investor	Range PDF report.
`/api/investor-outreach/send`	POST	Super Admin	Send outreach email.
`/api/investor-outreach/log`	GET	Super Admin	Outreach history.
`/api/legal-documents`	CRUD	Super Admin, Investor (scoped)	Legal doc storage.

Messaging

VERB	PATH	ROLE	PURPOSE
POST	`/api/messages`	any	Send message.
GET	`/api/messages`	any	List messages.
GET	`/api/messages/:driverName`	any	Conversation with driver.
PUT	`/api/messages/read`	any	Mark read.
PUT	`/api/notifications/read`	any	Mark notification read.
`/api/dispatch-notifications`	GET	Super Admin, Dispatcher	Admin notifications.
`/api/investor/messages`	GET/POST	Super Admin, Investor	Investor chat.

Expenses & finance

VERB	PATH	ROLE	PURPOSE
POST	`/api/expenses`	any	Log expense.
POST	`/api/expenses/ocr`	Driver, Super Admin, Dispatcher	Gemini receipt OCR (rate-limited).
GET	`/api/expenses/all`	Super Admin, Dispatcher	All expenses.
PUT	`/api/expenses/:id/status`	Super Admin	Approve/reject.
GET	`/api/expenses/fuel-analytics`	Super Admin	Fuel spend analytics.
`/api/maintenance-fund`	CRUD	Super Admin	Maintenance fund.
`/api/compliance/fees`	CRUD	Super Admin	Compliance fees.
`/api/compliance/ifta`	GET	Super Admin	IFTA mileage by state.

Invoices

VERB	PATH	ROLE	PURPOSE
POST	`/api/invoices/generate`	Super Admin	Generate from load data.
GET	`/api/invoices`	Super Admin, Driver (own)	List invoices.
GET	`/api/invoices/:id/pdf`	Super Admin, Driver (own)	Download invoice PDF.
PUT	`/api/invoices/:id/submit`	Driver, Dispatcher	Submit for approval.
PUT	`/api/invoices/:id/approve`	Super Admin	Approve.

Financials

VERB	PATH	ROLE	PURPOSE
GET	`/api/financials`	Super Admin	Aggregated P&L.

Public tracker

VERB	PATH	ROLE	PURPOSE
GET	`/api/public/track/:loadId`	public	Customer-facing tracker (rate-limited, response-whitelisted).

Documents & uploads

VERB	PATH	ROLE	PURPOSE
POST	`/api/documents/upload`	any	Upload POD/document to Drive.
GET	`/api/documents/:loadId`	any	List documents for load.
POST	`/api/chat/attachment`	any	Upload chat attachment.
POST	`/api/legal-documents/upload`	Super Admin	Upload truck/investor/driver legal doc.
PATCH	`/api/legal-documents/:id/visibility`	Super Admin	Toggle driver visibility.
GET	`/api/legal-documents?truck_id=`	Super Admin, Investor (scoped)	Scoped fetch.

Location & maps

VERB	PATH	ROLE	PURPOSE
POST	`/api/location`	Driver	Report GPS position.
GET	`/api/locations/latest`	Super Admin, Dispatcher	Latest position per active driver.
GET	`/api/locations/trail`	Super Admin, Dispatcher	Historical GPS trail.
GET	`/api/route`	any	Route directions.
`/api/geocode`	GET	any	Geocode address (cached).
`/api/geocode/search`	GET	any	Places search.
`/api/geocode/bulk`	POST	Super Admin	Batch geocode.
`/api/geocode/load/:loadId`	GET	any	Geocode a specific load's addresses.
GET	`/api/config/maps-key`	any	Expose Maps API key to frontend.
GET	`/api/weather`	any	Weather for coordinates.

Admin tools

VERB	PATH	ROLE	PURPOSE
GET	`/api/admin/audit-trail`	Super Admin	View audit log.
PUT	`/api/admin/fix-driver-name`	Super Admin	Rename driver across sheets.
GET	`/api/admin/scan-duplicates`	Super Admin	Find duplicates.
GET	`/api/admin/scan-driver-mismatches`	Super Admin	Find mismatches.
GET	`/api/admin/scan-orphans`	Super Admin	Find orphans.
GET	`/api/admin/scan-stale-locations`	Super Admin	Stale GPS.
POST	`/api/admin/fix-stale-locations`	Super Admin	Clean stale.
POST	`/api/admin/remove-rows`	Super Admin	Batch remove.
GET	`/api/archive`	Super Admin	Archived data.
GET	`/api/archive/tabs`	Super Admin	Archive tabs list.

Database admin

VERB	PATH	ROLE	PURPOSE
GET	`/api/db/download`	Super Admin	Download SQLite file.
GET	`/api/db/tables`	Super Admin	List tables.
GET	`/api/db/query/:table`	Super Admin	Query a table.

Routemate (telematics)

VERB	PATH	ROLE	PURPOSE
POST	`/api/admin/routemate/sync-now`	Super Admin	Manual vehicle sync.
GET	`/api/routemate/health`	Super Admin	Health probe.

Webhooks

VERB	PATH	AUTH	PURPOSE
POST	`/api/n8n/job`	`x-webhook-secret`	New load from n8n.
POST	`/api/webhook/new-load`	`x-webhook-secret`	Secondary ingestion path.

Debug (development)

VERB	PATH	ROLE	PURPOSE
GET	`/api/debug/driver-view/:driverName`	Super Admin	What this driver sees.
GET	`/api/debug/driver-empty/:driverName`	Super Admin	Empty-state probe.
GET	`/api/debug/sample-row`	Super Admin	First sheet row.
GET	`/api/debug/driver-loads/:driverName`	Super Admin	Driver's loads, raw.
GET	`/api/debug/user/:username`	Super Admin	User record dump.

Socket.IO events

EVENT	DIRECTION	ROOM
`register`	client → server	—
`load-assigned`	server → client	`driver:<name>`
`load-cancelled`	server → client	`driver:<name>`
`load-deleted`	server → client	`dispatch`
`load-accepted` / `load-declined`	server → client	`dispatch`
`status-updated`	server → client	`dispatch`
`dispatch-notification`	server → client	`dispatch`
`new-message`	server → client	all
`pod-uploaded`	server → client	`dispatch`
`location-update`	server → client	`dispatch`
`geofence-trigger`	server → client	`dispatch`, `driver:<name>`
`reload`	server → client	all